Scuola universitaria professionale della Svizzera italiana
Dipartimento tecnologie innovative

SUPSI

3D Terrain Generator Generazione procedurale e editing interattivo di terreni 3D con OpenGL

Studente/i
Pietro Brusadelli
Giona Valsecchi

Relatore
Randolf Schärfig

Correlatore
Achille Peternier

Committente
Schärfig Randolf

Corso di laurea
Ingegneria Informatica

Codice progetto
C10981

Anno
2025 – 2026

Data [Data]

STUDENTSUPSI

Indice generale

1.1 Il punto di partenza: l'engine preesistente **5**
1.1.1 Componenti dell'engine fornito 5

1.2 Analisi bibliografica **6**
1.2.1 Tecnologie disponibili 6

2.1 Obiettivi del progetto **7**

2.2 Metodologia **7**
2.2.1 Scelte tecniche chiave 8

3.1 Struttura del sistema **9**

3.2 Sistema di editing **11**

3.3 Sistema di esportazione: **12**

4.1 Verifica degli obiettivi **14**

4.2 Analisi delle prestazioni **14**

4.3 Confronto con approcci alternativi **14**

4.4 Limiti identificati **15**

A — Diagrammi UML **17**
A1 — Architettura generale (MVC) 17
A2 — Gerarchia delle finestre (Template Method) 18
A3 — Gerarchia dei brush (Template Method) 19
A4 — Gerarchia dei tool (Strategy) 20
A5 — EventDispatcher (Observer) 20
A6 — Sequence diagram: generazione del terreno 21

Allegato A — Benchmark delle prestazioni **24**

STUDENTSUPSI

Riassunto / Abstract

Il presente rapporto descrive lo sviluppo del 3D Terrain Generator, un'applicazione desktop per la generazione e l'editing in tempo reale di un terreno tridimensionale. Il progetto è sviluppato a partire da un engine OpenGL già iniziato, che è stato sviluppato nel corso di Grafica del semestre scorso e successivamente rielaborato, seguendo il corso di Realtà Virtuale, con un nuovo sistema di shader, di illuminazione e di LOD.La parte di client è stata sviluppata da noi per questo progetto. Le funzionalità principali comprendono: La generazione della heightmap tramite Perlin Noise con parametri interamente configurabili, il rendering real-time della mesh 3D con sistema LOD per non appesantire, pennelli differenti per la modifica del terreno (sculpting, erosion e smoothing), più modalità differenti di visualizzazione (realistic, solid, wireframe, mappa altimetrica) e l'esportazione del terreno nei formati OBJ (per la mesh) ed EXR (per la heightmap). La scelta architetturale centrale consiste nell'applicare il displacement della mesh direttamente nel vertex shader a partire dalla heightmap memorizzata come texture GPU: questa soluzione fa in modo che la texture sia modificabile in tempo reale anche su mesh di grandi dimensioni, dato che ogni modifica richiede il solo aggiornamento della parte di texture richiesta tramite glTexSubImage2D, senza ricaricare la geometria. Il risultato è un sistema funzionante, capace di gestire terreni di NxN vertici mantenendo frame rate buoni grazie al LOD, che riduce il numero di triangoli renderizzati per i chunk più distanti.

This report describes the development of the 3D Terrain Generator, a desktop application for the real-time generation and editing of a three-dimensional terrain. The project is built upon a pre-existing OpenGL engine, originally developed during last semester's Graphics course and subsequently reworked alongside the Virtual Reality course to include a new shader, lighting, and LOD (Level of Detail) system. Everything except the pre-existing engine was developed during this semester.

The main features include: heightmap generation using Perlin Noise with fully configurable parameters, real-time rendering of the 3D mesh with a LOD system to optimize performance, various brushes for terrain modification (sculpting, erosion, and smoothing), multiple display modes (realistic, solid, wireframe, and heightmap), and terrain export in OBJ (for the mesh) and EXR (for the heightmap) formats.

The core architectural choice consists of applying mesh displacement directly in the vertex shader using the heightmap stored as a GPU texture. This solution ensures that the texture can be modified in real-time even on large meshes, since each modification only requires updating the affected portion of the texture via glTexSubImage2D, without the need to reload the geometry. The result is a functional system capable of handling terrains of NxN vertices while maintaining good frame rates thanks to the LOD, which reduces the number of rendered triangles for more distant chunks.

Progetto assegnato

Descrizione:

- Nell'ambito del progetto si richiede di sviluppare un sistema di generazione di terreni basato su OpenGL. Lo scopo è quello di ottenere un terreno 3D realistico, inizialmente generato a partire dall'algoritmo di Perlin Noise. L'utente dovrà poter modificare direttamente il terreno in 3D tramite degli strumenti, creando colline e valli con il mouse. Il programma deve inoltre permettere all'utente di esportare il terreno 3D nel formato .OBJ.

Compiti:

- Creare e visualizzare mesh 3D di grandi dimensioni.
- Generare un'immagine di Perlin Noise e utilizzarla per costruire la struttura iniziale del terreno.
- Implementare l'interazione tramite mouse per consentire l'editing del terreno iniziale.

Obiettivi:

- Estendere un engine esistente in modo da supportare il rendering di terreni di grandi dimensioni.
- Ottimizzare il codice per garantire frame-rate ragionevoli.
- OpenGL, C++.

STUDENTSUPSI

Capitolo 1

1.1 Il punto di partenza: l'engine preesistente

Il requisito principale del progetto era di estendere un engine implementato con OpenGL. L'engine fornito è stato creato durante il corso di Grafica seguito nel semestre scorso. Esso è stato migliorato grazie alle nozioni apprese durante la prima metà del corso di Realtà Virtuale. La parte di Client descritta nei capitoli successivi è invece interamente sviluppata da noi per il 3D Terrain Generator.

1.1.1 Componenti dell'engine fornito

L'engine fornisce una struttura di rendering basata su uno scene graph gerarchico a nodi (parent-child), che permette di organizzare gli oggetti 3D nella scena in modo strutturato.

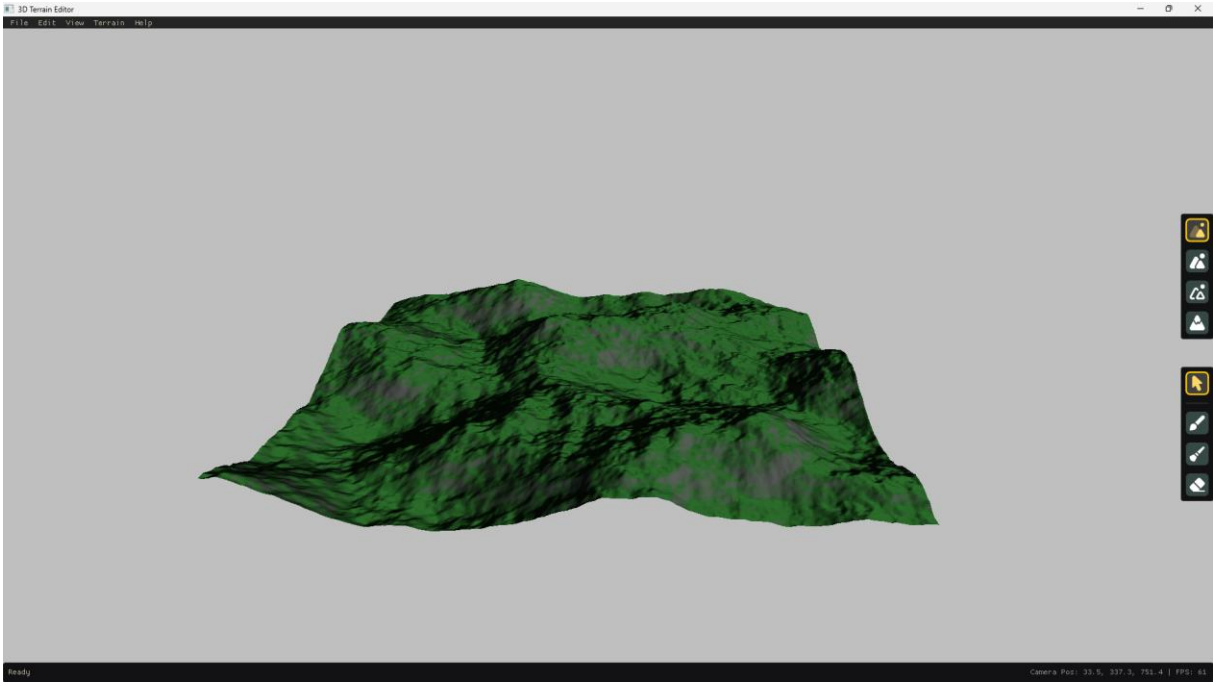


Figura 1: [Screenshot della vista principale dell'applicazione]

1.2 Analisi bibliografica

La scelta delle funzioni per la generazione dell'immagine del Perlin Noise sono state scelte dopo la consultazione del documento README del repository GitHub PerlinNoise di Reputeless [1] (https://github.com/Reputeless/PerlinNoise). Per quanto riguarda le scelte fatte dal punto di vista di generazione della mesh e renderizzazione, ci siamo basati sulla documentazione e le lezioni del professor Achille Peternier al corso di Realtà Virtuale [2]–[5] (vr ogl4 efficientData File , vr ogl4 shaders File , vr ogl4 ppl File , vr virtualEnvironments File)

1.2.1 Tecnologie disponibili

Sulla base dell'engine iniziale e dei requisiti della consegna, abbiamo utilizzato:

- C++17 come linguaggio principale
- OpenGL 4.4 come API di rendering 3D
- GLSL 4.4 per i vertex e fragment shader
- FreeGLUT per il window management e il contesto OpenGL
- GLEW come OpenGL Extension Wrangler
- TinyEXR [6] per l'esportazione della heightmap in formato EXR
- Dear ImGui [7] per l'interfaccia utente
- GLM per la matematica vettoriale e matriciale
- La libreria header-only PerlinNoise di Ryo Suzuki (Reputeless) per l'implementazione del rumore
- nfd (Native File Dialog) [8] per i dialoghi di salvataggio

Capitolo 2

2.1 Obiettivi del progetto

Abbiamo definito gli obiettivi del lavoro mettendoli in ordine di priorità. Gli obiettivi principali sono:

- implementare la creazione di una heightmap tramite Perlin Noise con parametri interamente personalizzabili (seed, frequenza, numero di ottave e dimensione).
- Costruire una mesh a partire dalla heightmap, possibilmente suddivisa in chunk per avere prestazioni accettabili.
- Sviluppare shader dedicati per il rendering del terreno con displacement dalla heightmap, calcolo delle normali.
- Implementare strumenti di editing per il terreno tramite pennello (sculpting, erosione, smoothing) in tempo reale.
- Consentire l'esportazione del terreno nel formato OBJ.

Come obiettivi secondari abbiamo identificato:

- Fornire più modalità di visualizzazione (realistico, solid, wireframe, mappa altimetrica)
- Supportare la navigazione 3D intuitiva che somigli a editor già esistenti (pan, rotazione, zoom).
- Esportare la heightmap come file EXR

Come obiettivi extra implementabili:

- Lavorare il terreno tramite l’utilizzo del Leap Motion

2.2 Metodologia

Abbiamo affrontato il progetto partendo dalla generazione della heightmap per testarne le funzionalità. In seguito abbiamo implementato la generazione della mesh, collegandole infine assieme per visualizzare il terreno a schermo. Dopo di ciò abbiamo implementato quanto riguarda i pennelli e l’esportazione del .OBJ.

La scelta architetturale centrale è stato l’utilizzo del pattern Model-View-Controller (MVC): il Model contiene i dati del terreno (heightmap, chunk mesh, configurazioni), la View è costituita dalle finestre ImGui che presentano i dati all'utente e permettono l’inserimento dei parametri.I Controller infine mediano tra Model e View e coordinano le operazioni.

Questa separazione facilita la manutenibilità: aggiungere un nuovo strumento di editing richiede solo una nuova sottoclasse del Brush, senza dover modificare troppe componenti. Sono inoltre stati usati altri pattern: Singleton per i controller globali (AppController, SetupController, …); Strategy per i brush a partire da BaseBrushTool, Template Method per la pipeline comune di editing in BaseBrushTool::use(), Observer tramite un EventDispatcher per il decoupling tra componenti (i diagrammi UML di questi pattern sono riportati nell'Appendice A).Infine il pattern Factory per il metodo TerrainModel::generateTerrain().

STUDENTSUPSI

2.2.1 Scelte tecniche chiave

Aggiornamento della heightmap

Abbiamo scelto di mantenere la geometria della mesh statica e applicare le altezze esclusivamente nel vertex shader, tramite texture lookup sulla heightmap, invece di aggiornare le posizioni dei vertici. La motivazione è evitare un calo di prestazioni. Infatti aggiornare solo la regione modificata della texture heightmap sulla GPU (utilizzando la funzione glTexSubImage2D) è più veloce che riscrivere e ricaricare tutti i vertici della mesh dopo ogni operazione di editing. Inoltre, le normali calcolate nel vertex shader sono sempre coerenti con la heightmap corrente, eliminando la necessità di ricalcolarle a parte.
Funzionamento:

1. PointerController.update() controlla tramite InputController se il terreno è “colpito” dal puntatore.
2. Il punto colpito viene tradotto in una posizione sulla heightmap nel metodo use() di BaseBrushTool(), che in seguito itera su ogni pixel e applica l’effetto a seconda del brush, restituendo un UpdateArea.
3. Il PointerController controlla se la UpdateArea è cambiata, in quel caso chiama updateSubImage per aggiornare solo la parte interessata.
4. Al frame successivo il vertex shader legge i nuovi valori, ricalcolando le normali.

Utilizzo dei LOD:

Invece di creare la mesh del terreno come un unico oggetto, la funzione TerrainGenerator::generate() la suddivide in una griglia di chunk indipendenti. La dimensione di ogni chunk è al massimo 64x64 vertici: se il terreno ha meno di 64 vertici per lato il chunk si riduce di conseguenza. Nella configurazione standard (size=512, spacing=1.0) si ottengono 8x8 = 64 chunk da 64x64 vertici ciascuno.

Ogni chunk viene costruito da TerrainGenerator::generateChunk(), che produce:

- i vertici con Y=0 (il displacement in altezza avviene nel vertex shader)
- le coordinate UV calcolate rispetto alla heightmap globale, non al chunk locale, in modo che ogni vertice campioni il texel corretto della texture.
- quattro index buffer, uno per ogni livello LOD

I buffer vengono caricati su GPU da Mesh::initBuffers() al primo frame in cui il chunk viene renderizzato.

STUDENTSUPSI

Capitolo 3

3.1 Struttura del sistema

La parte Client è organizzata come insieme di moduli che si appoggiano sull'engine Eng sviluppato durante il semestre scorso al modulo di grafica (vedi Appendice A1 per il diagramma architetturale complessivo).

I moduli principali sono:

- TerrainModel (integrazione e ciclo di vita del terreno).
- TextureGenerator (generazione heightmap tramite Perlin Noise).
- TerrainGenerator (costruzione mesh e index buffer LOD).
- Un insieme di Brush Tools (SculptingBrushTool, ErosionBrushTool, SmoothingBrushTool).
- Un insieme di Controller (AppController, SetupController, PointerController, CameraGestureController, InputController, UIController, VisualController).
- Le View ImGui (SetupWindow, PointerToolWindow, PointerToolSettingsWindow, VisualToolWindow, TopMenuBar, StatusBar).
- Gli Exporter (ObjExporter, ImageExporter).
- Gli shader dedicati (terrain.vert, terrain.frag).

La heightmap è memorizzata come vettore di float organizzato in tre canali (R, G, B) per pixel.

L'altezza normalizzata nell'intervallo [0.0, 1.0] è memorizzata nei canali.

Essa viene generata utilizzando la libreria header-only PerlinNoise di Ryo Suzuki (Reputeless), combinando un macro noise (peso 0.85) e un micro noise multi-ottava a frequenza quadrupla (peso 0.15).

La correzione di potenza pari a 1.5 è stata inserita per poter essere parametrizzata alla creazione, così da poter generare dei terreni molto piani con delle montagne molto ripide, ma poi scartata in quanto non richiesta (rimane comunque implementabile in futuro).

I parametri esposti in UI sono size, frequency, octaves e seed, con valori di default rispettivamente 512, 4.0, 6 e 12345.

Il TerrainGenerator converte la heightmap in una mesh 3D suddivisa in chunk di 64×64 vertici (per un terreno 512×512 si ottengono 64 chunk in una griglia 8×8).

Ogni chunk memorizza quattro index buffer corrispondenti ai quattro livelli LOD, con soglie di attivazione rispettivamente a distanze di 500, 1000 e 2000 unità dalla camera. Per poter controllarne il funzionamento basta generare un terreno con frequenza e ottave minime ed entrare in modalità wireframe, in modo da poter controllare la differenza nella quantità di triangoli man mano che ci si distanzia dalla camera (vedi Figura 2).

STUDENTSUPSI

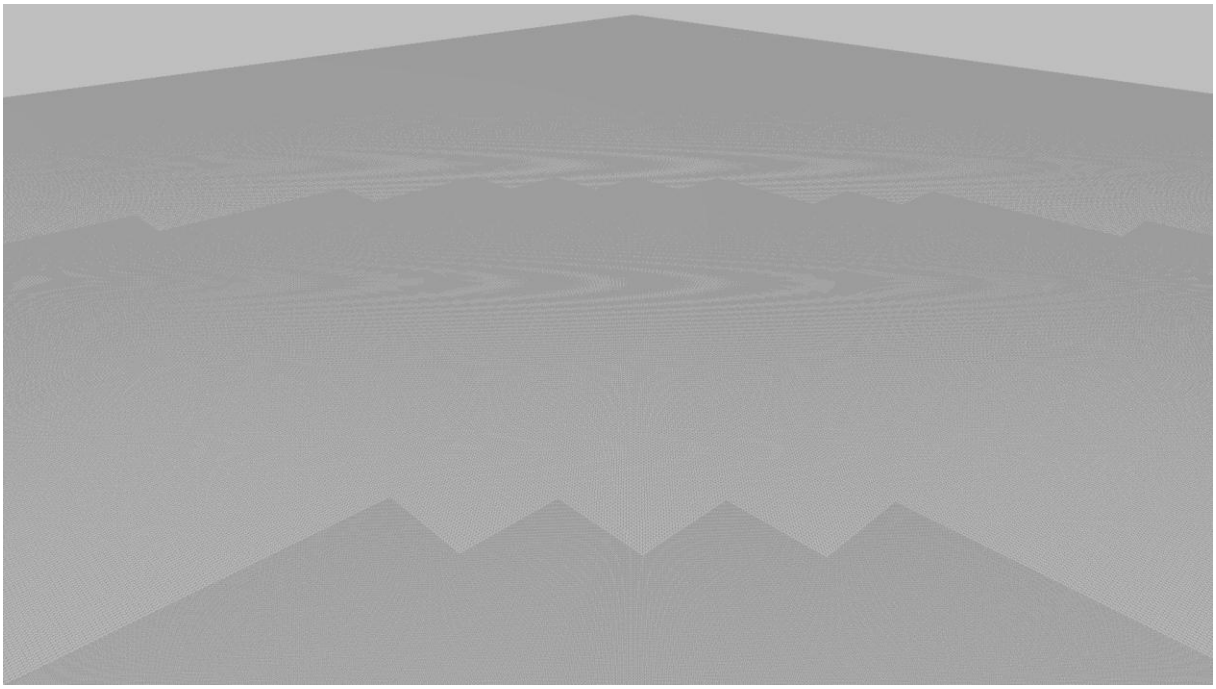


Figura 2: [Screenshot che mostra la differenza creata dai LOD]

Le coordinate Y dei vertici sono inizializzate a zero, lo spostamento verticale avviene nel vertex shader tramite displacement mapping.

Il vertex shader legge l'altezza dalla heightmap con texture lookup:

$$(\mathrm{y} = \mathrm{texture}(\mathrm{heightMap}, \mathrm{uv}).\mathrm{r} * \mathrm{heightScale})$$

e calcola le normali campionando quattro texel adiacenti (sinistra, destra, sopra, sotto). Questo garantisce che le normali siano corrette in base alla heightmap corrente, anche dopo le operazioni di editing, senza che la CPU debba ricalcolarle.

Il fragment shader terrain.frag supporta quattro modalità di visualizzazione selezionabili in UI.

- In modalità Shading Realistico il colore base è determinato dalla pendenza della superficie (dot tra normale e asse verticale) ossia le zone pianeggianti assumono un colore verde (erba), quelle ripide un colore grigio (roccia),una miscela delle due nella zona di transizione.
- L'illuminazione segue il modello di Blinn-Phong con una luce direzionale che simula il sole.
- Poiché le normali vengono ricalcolate nel vertex shader a ogni frame a partire dalla heightmap corrente (vedi par. precedente), l'illuminazione e le ombre prodotte dal modello restano sempre coerenti con la geometria visibile anche durante l'editing in tempo reale: ogni modifica al terreno aggiorna la heightmap, il vertex shader rilegge i texel adiacenti e ricalcola le normali, e il fragment shader applica Blinn-Phong sulle normali aggiornate. Il risultato è che il terreno reagisce alla luce in modo realistico fotogramma per fotogramma, senza alcun ricalcolo lato CPU.
- La modalità Solid mostra un colore uniforme, utile per verificare la forma della mesh
- La modalità Wireframe visualizza i soli spigoli, utile per analizzare la struttura e il LOD.
- La modalità Altimetrica applica un gradiente blu, verde, rosso basato sull'altezza normalizzata.
- In tutte le modalità, quando un brush è attivo, lo shader disegna un cerchio nella posizione del cursore fornendo feedback sull'area dove editeremo il terreno.

STUDENTSUPSI

3.2 Sistema di editing

Tutti i tool di editing del terreno ereditano dalla classe base BaseBrushTool secondo il pattern Template Method (vedi Appendice A3).

La gerarchia è la seguente:

- BaseTool
    - BaseBrushTool
        - SculptingBrushTool (alza il terreno)
        - ErosionBrushTool (abbassa il terreno)
        - SmoothingBrushTool (leviga il terreno).

La pipeline in BaseBrushTool::use() è composta da cinque passi:

1. ricezione delle coordinate 3D del mondo, già risolte da PointerController tramite ray casting sul piano del terreno e conversione in coordinate UV della texture tramite normalizzazione rispetto alla dimensione fisica del terreno.

$$float\ localX\ =\ coords.x\ +\ (terrainPhysicalSize\ /\ 2.0f);$$

$$float\ localZ\ =\ coords.z\ +\ (terrainPhysicalSize\ /\ 2.0f);$$

$$float\ u\ =\ localX\ /\ terrainPhysicalSize;$$

$$float\ v\ =\ localZ\ /\ terrainPhysicalSize;$$

$$int\ pixelX\ =\ static_cast < int > (u\ *\ imageResolution);$$

$$int\ pixelY\ =\ static_cast < int > (v\ *\ imageResolution);$$

2. calcolo dell'area affetta in pixel, con pixelRadius = radius × (imageResolution / terrainSize)
3. Per ogni pixel nell'area, chiamata a applyBrushEffect(), che contiene la logica del tool a dipendenza di come vogliamo lavorare il terreno
4. Upload della regione modificata sulla GPU tramite glTexSubImage2D.
5. Al termine del ciclo di modifica, `BaseBrushTool::use` restituisce una struct `UpdateArea` contenente i bounds del rettangolo di pixel toccato (`startX`, `startY`, `width`, `height`) e un flag `isModified`; questi valori verranno usati a monte per aggiornare solo la sotto-regione corrispondente della texture GPU. Il falloff del pennello è modellato tramite il polinomio di Hermite cubico dato da glm::smoothstep
    a. Calcola la distanza (in pixel) tra il pixel corrente (x, y) e il centro del pennello (pixelX, pixelY)

$$distance\ =\ glm::distance(pixel, centro)$$

    b. Normalizza la distanza in [0, 1]: t = 0 al centro, t = 1 sul bordo del cerchio.

$$t\ =\ distance\ /\ pixelRadius$$

STUDENTSUPSI

c. smoothstep è una curva a S che vale 0 in t=0 e 1 in t=1, con derivate nulle agli estremi (transizione morbida). Sottraendo da 1 la si inverte: smooth = 1 al centro, scende a 0 sul bordo seguendo una curva morbida.

$$smooth \quad = 1 - glm :: smoothstep(0, 1, t)$$

d. Abbiamo le seguenti possibilità:

   i. Se falloff = 0 → influence = 1 ovunque (pennello "duro", uniforme dentro al cerchio).

   ii. Se falloff = 1 → influence = smooth (pennello completamente sfumato).

   Valori intermedi → interpolazione tra i due comportamenti.

$$influence = glm::mix(1.0, smooth, falloff)$$

3.3 Sistema di esportazione:

ObjExporter converte la heightmap in una mesh 3D nel formato Wavefront OBJ.

1. Lettura dell'altezza dalla heightmap in getH:

   In questa lambda vengono passate le coordinate UV e viene restituita l'altezza in quel punto. Il clamp serve per fare in modo che, quando si calcolano le normali, non si accedano a pixel fuori dall'area.

$$int\ x = std::clamp(int(u * (size - 1)), 0, size - 1);$$

   Nella parte successiva si accede alla heightmap all'indice calcolato con

$$image[(y * size + x) * 3]$$

   La moltiplicazione per 3 è necessaria perché la heightmap memorizza tre canali (R, G, B) per pixel in un array lineare di float: l'offset del pixel (x, y) si ottiene moltiplicando l'indice lineare per 3. Viene letto il canale R, che contiene l'altezza normalizzata nell'intervallo [0.0, 1.0].

2. Iterazione sui chunk e scrittura delle informazioni:

   a. Per ogni chunk vengono presi vertici, coordinate UV e LOD.

$$const\ auto\&\ vertexes = chunk{-}{>}\ getVertexes();$$

$$const\ auto\&\ texCoords = chunk{-}{>}\ getTextureCoordinates();$$

$$const\ auto\&\ faces = chunk{-}{>}\ getFacesForLOD(activeLod);$$

   b. Vengono in seguito recuperate le altezze per la scrittura del vertice completo in tre dimensioni

$$float\ h = getH(texCoords[i].x, texCoords[i].y);$$

$$file << "v" << vertexes[i].x << " " << h << " " << vertexes[i].z << "\backslash n";$$

STUDENTSUPSI

c. Vengono scritte le coordinate UV:

$$file << "vt" << vt.x << " " << (1.0f - vt.y) << "\n";$$

d. Vengono calcolate le normali facendo le differenze finite centrate, inizialmente si leggono i 4 punti attorno al vertice centrale:

$$float\ hL = getH(vt.x - texelSize, vt.y);$$

$$float\ hR = getH(vt.x + texelSize, vt.y);$$

$$float\ hD = getH(vt.x, vt.y - texelSize);$$

$$float\ hU = getH(vt.x, vt.y + texelSize);$$

In seguito vengono trovate le pendenze X e Z:

$$float\ nx = hL - hR;$$

$$float\ nz = hD - hU;$$

Successivamente viene trovata anche la componente Y, normalizzata moltiplicandola per il parametro heightScale.

$$float\ ny = 2.0f * texelSize * heightScale;$$

e. Infine vengono inserite le facce. Il +1 viene impiegato in quanto nel formato .OBJ gli indici partono da 1 e non da 0.

$$unsigned\ int\ i1 = face.x + 1 + globalVertexOffset;$$

$$unsigned\ int\ i2 = face.y + 1 + globalVertexOffset;$$

$$unsigned\ int\ i3 = face.z + 1 + globalVertexOffset;$$

Viene poi aggiornato globalVertexOffset per mantenere il valore dell'indice anche al cambio di chunk.

$$globalVertexOffset += vertexes.size();$$

STUDENTSUPSI

Capitolo 4

4.1 Verifica degli obiettivi

Tutti gli obiettivi definiti nel Capitolo 2 eccetto la modifica tramite Leap Motion sono implementati. La generazione Perlin Noise è configurabile per seed, frequenza, ottave e dimensione.
La mesh generata a chunks dispone di LOD a quattro livelli.
Lo shader applica displacement, calcola normali dinamiche e le quattro modalità di visualizzazione previste.
I tre brush (sculpting, erosione, smoothing) sono completi e i parametri raggio, strength e falloff sono tutti configurabili da UI.
L'export è disponibile sia in OBJ (asincrono, con progress bar) sia in EXR.
La navigazione 3D (pan, rotazione, zoom) funziona con il mouse.

4.2 Analisi delle prestazioni

L'impatto del sistema LOD sul numero di poligoni è sostanziale.
Per un terreno di 512×512 vertici, senza LOD, il numero totale di triangoli sarebbe $511 \times 511 \times 2 \approx 522.000$.
Con il sistema LOD attivo e camera a distanza variabile, il numero effettivo di triangoli renderizzati si riduce significativamente per i chunk distanti.
Il sistema LOD permette quindi di visualizzare terreni di grandi dimensioni mantenendo frame rate accettabili anche su hardware di fascia media. I dati di benchmark completi sui tempi di generazione sono riportati nell'Allegato A.
L'approccio di displacement via shader elimina qualsiasi aggiornamento della geometria in RAM dopo le operazioni di editing, l'unica operazione necessaria è un glTexSubImage2D sulla regione modificata, che in pratica risulta trascurabile anche per pennelli con un raggio molto grande.

4.3 Confronto con approcci alternativi

Generazione del noise. La libreria PerlinNoise di Reputeless è una delle implementazioni C++17 più diffuse e performanti, ed era la scelta naturale dato che l'uso del Perlin Noise era richiesto dalla consegna.
Il Perlin Noise 2D produce risultati visivamente soddisfacenti per il nostro caso d'uso.
Gestione della heightmap. L'alternativa al displacement via shader sarebbe stata la modifica diretta dei vertici della mesh con successivo re-upload.
Questa soluzione avrebbe comportato costi di upload maggiori ad ogni modifica del pennello.

STUDENTSUPSI

4.4 Limiti identificati

Il sistema presenta alcuni limiti che abbiamo identificato a valle della validazione.
Assenza di undo/redo: ogni modifica al terreno è permanente, e la sua implementazione richiederebbe di mantenere in memoria uno storico delle modifiche della heightmap.
Assenza di salvataggio del progetto, non è possibile salvare lo stato di lavoro corrente (heightmap + configurazione) e riaprirlo in una sessione successiva; l'unica forma di persistenza è l'export in OBJ/EXR, che però non può essere ricaricato come progetto editabile.
Supporto Leap Motion non implementato: l'obiettivo extra di editing tramite Leap Motion non è stato realizzato per ragioni di tempo.

Conclusioni

Il lavoro ha prodotto un'applicazione desktop completa per la generazione procedurale e l'editing interattivo di terreni 3D, che soddisfa tutti i requisiti definiti nella consegna del progetto.
In particolare, la scelta di eseguire lo spostamento geometrico interamente nel vertex shader è la decisione più impattante in termini di prestazioni, poiché rende possibile l'editing interattivo in tempo reale anche su mesh di grandi dimensioni senza dover rigenerare la geometria ad ogni modifica.
Dal punto di vista della generalizzabilità, l'architettura a brush e a plugin sviluppata per i tool di editing è facilmente estendibile, infatti se si vuole aggiungere un nuovo strumento (per esempio un tool che uniforma il terreno a un'altezza target) richiede solo la creazione di una nuova sottoclasse di BaseBrushTool con la propria implementazione di applyBrushEffect(), senza modificare alcun altro componente del sistema.

Bibliografia

- I riferimenti sono citati nel testo tra parentesi quadre [n] e organizzati qui per tipologia di fonte.
-
- Repository software
- [1] R. Suzuki (Reputeless), "PerlinNoise: Header-only Perlin noise library for modern C++", GitHub repository. [Online]. Disponibile: https://github.com/Reputeless/PerlinNoise
- Dispense del corso
- [2] A. Peternier, "Virtual Reality — OpenGL 4: (more) efficient data", dispensa del corso di Realtà Virtuale, SUPSI, Dipartimento Tecnologie Innovative, a.a. 2025–2026.
- [3] A. Peternier, "Virtual Reality — OpenGL 4: shaders", dispensa del corso di Realtà Virtuale, SUPSI, Dipartimento Tecnologie Innovative, a.a. 2025–2026.
- [4] A. Peternier, "Virtual Reality — OpenGL 4: per-pixel lighting", dispensa del corso di Realtà Virtuale, SUPSI, Dipartimento Tecnologie Innovative, a.a. 2025–2026.

- [5] A. Peternier, "Virtual Reality — Virtual environments", dispensa del corso di Realtà Virtuale, SUPSI, Dipartimento Tecnologie Innovative, a.a. 2025–2026.
- Repository software (librerie di supporto)
- [6] S. Fujita (syoyo), "TinyEXR: Small, single header-only library to load and save OpenEXR (.exr) images", GitHub repository. [Online]. Disponibile: https://github.com/syoyo/tinyexr
- [7] O. Cornut, "Dear ImGui: Bloat-free Graphical User interface for C++ with minimal dependencies", GitHub repository. [Online]. Disponibile: https://github.com/ocornut/imgui
- [8] M. Labbe (mlabbe), "Native File Dialog: A tiny, neat C library that portably invokes native file open and save dialogs", GitHub repository. [Online]. Disponibile: https://github.com/mlabbe/nativefiledialog

STUDENTSUPSI

Appendici

A — Diagrammi UML

Questa appendice raccoglie i diagrammi UML dell'architettura del 3D Terrain Generator. I diagrammi documentano la struttura delle classi, la gerarchia dei pattern adottati (vedi 2.2) e il flusso di generazione del terreno descritto nei Capitoli 2 e 3.

A1 — Architettura generale (MVC)

Diagramma d'insieme dei moduli del sistema, organizzati secondo il pattern Model-View-Controller ed Engine. Mostra le dipendenze tra le finestre ImGui della View, i Controller singleton, il TerrainModel con i suoi generatori e gli utility di esportazione.

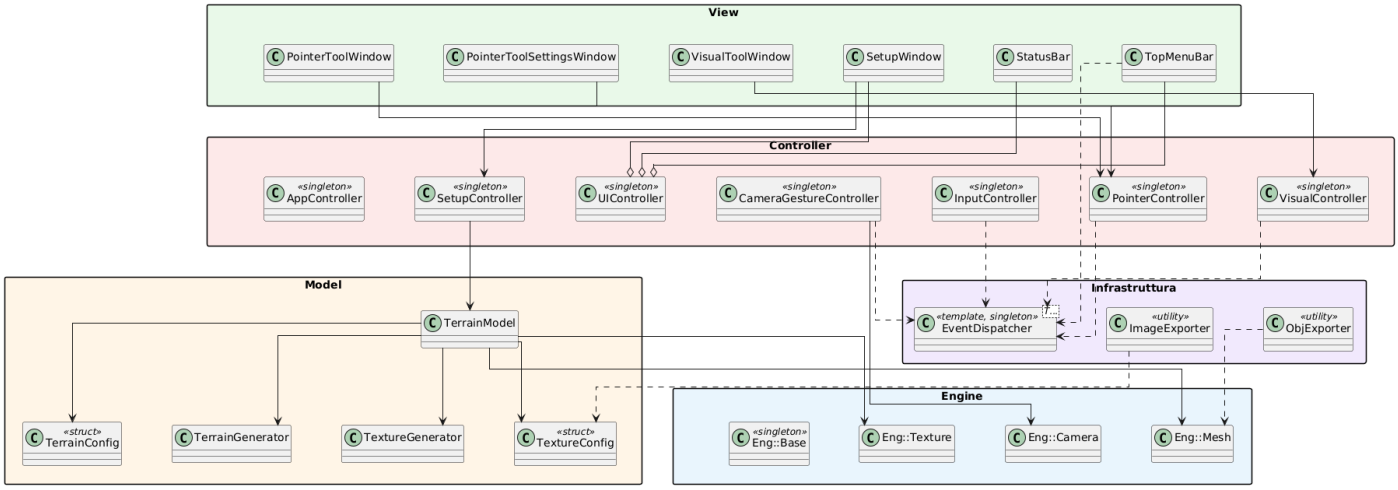


Figura A1: Architettura complessiva del 3D Terrain Generator (MVC).

STUDENTSUPSI

A2 — Gerarchia delle finestre (Template Method)

Gerarchia della classe astratta BaseWindow e delle sue sottoclassi.
Il metodo render() implementa il Template Method:

- calcola la posizione, apre il contesto ImGui, delega a drawContent() (hook ridefinito nelle sottoclassi concrete) e chiude il contesto.

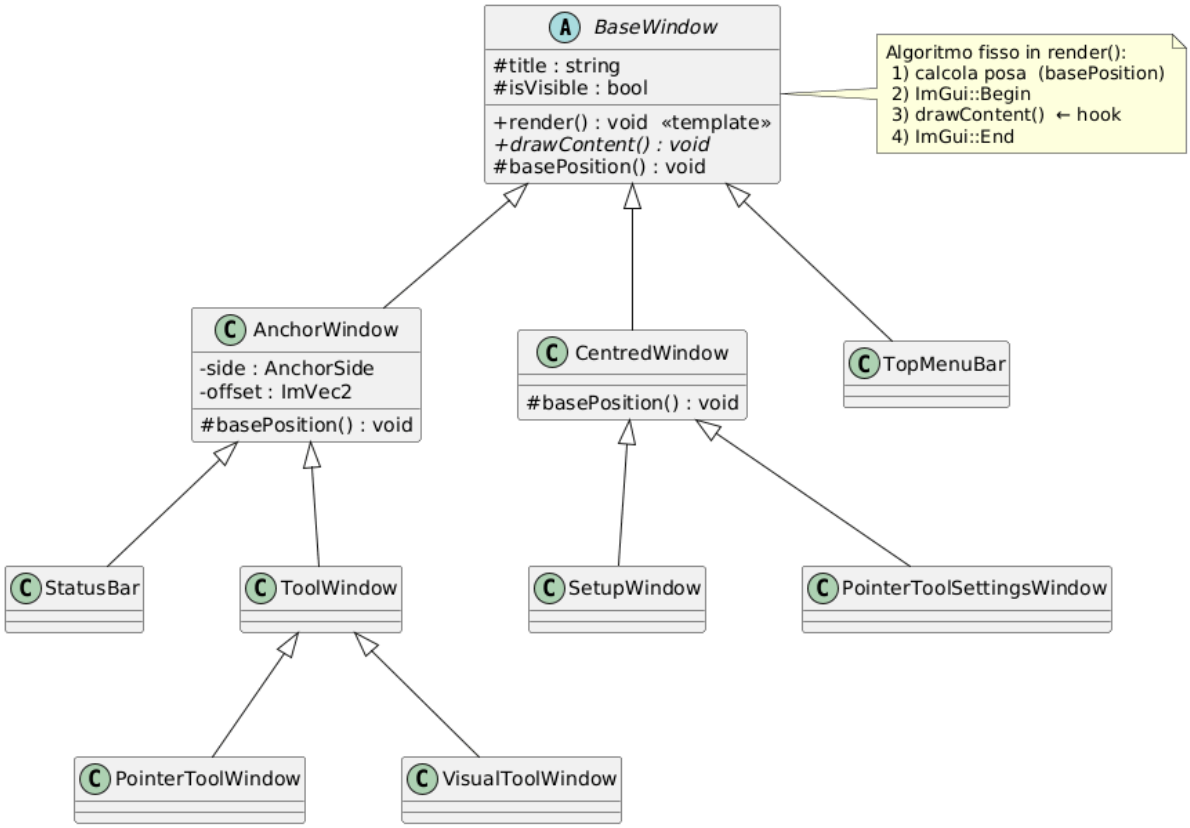


Figura A2: Gerarchia delle finestre e pattern Template Method in BaseWindow::render().

STUDENTSUPSI

A3 — Gerarchia dei brush (Template Method)

Gerarchia della classe astratta BaseBrushTool e delle tre sottoclassi concrete (SculptingBrushTool, SmoothingBrushTool, ErosionBrushTool).
Il metodo use() implementa il Template Method descritto in 3.2 ossia conversione delle coordinate, clipping al bordo dell'immagine, iterazione sui pixel con chiamata al hook applyBrushEffect(), restituzione dell'UpdateArea.

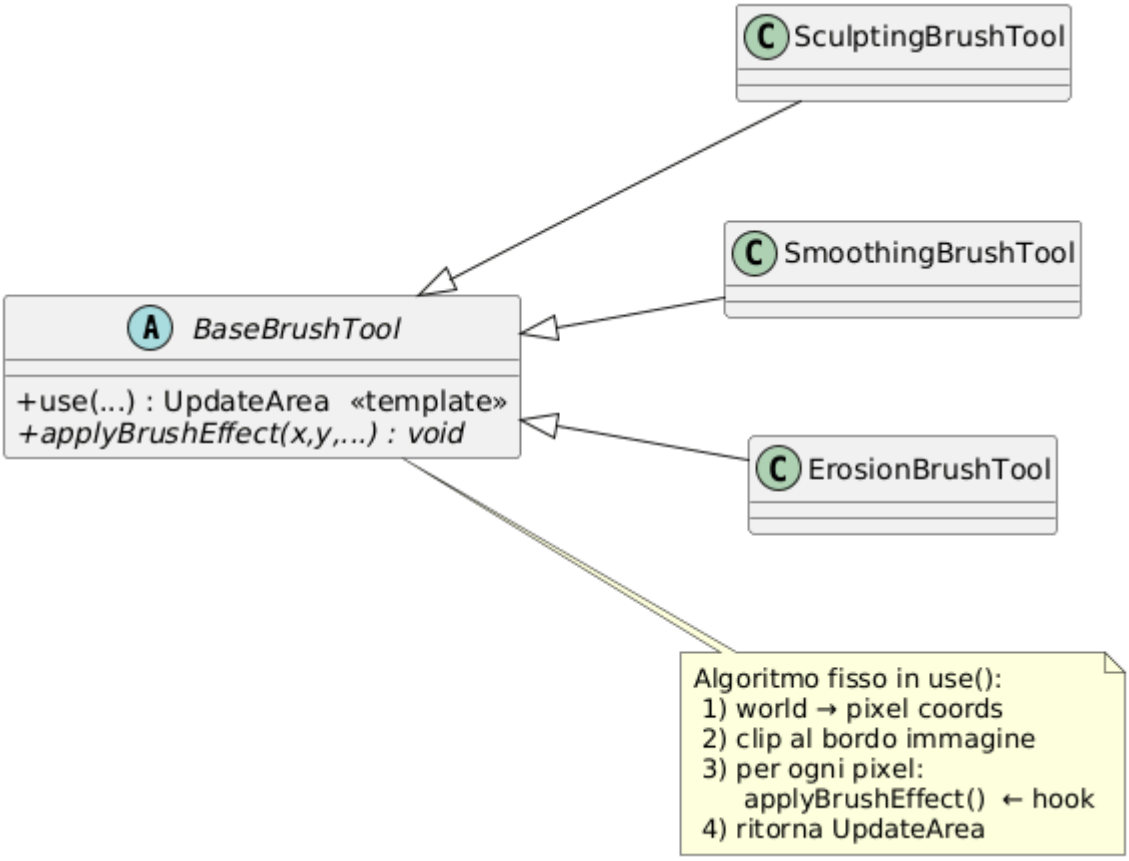


Figura A3: Gerarchia dei brush e pattern Template Method in BaseBrushTool::use().

STUDENTSUPSI

A4 — Gerarchia dei tool (Strategy)

Gerarchia BaseTool, con le due famiglie BaseBrushTool e BaseVisualTool e il CursorTool. I ToolController (PointerController, VisualController) detengono un puntatore al currentTool e ne delegano l'uso: applicare il pattern Strategy permette di cambiare il comportamento del tool attivo a runtime senza modificare il controller.

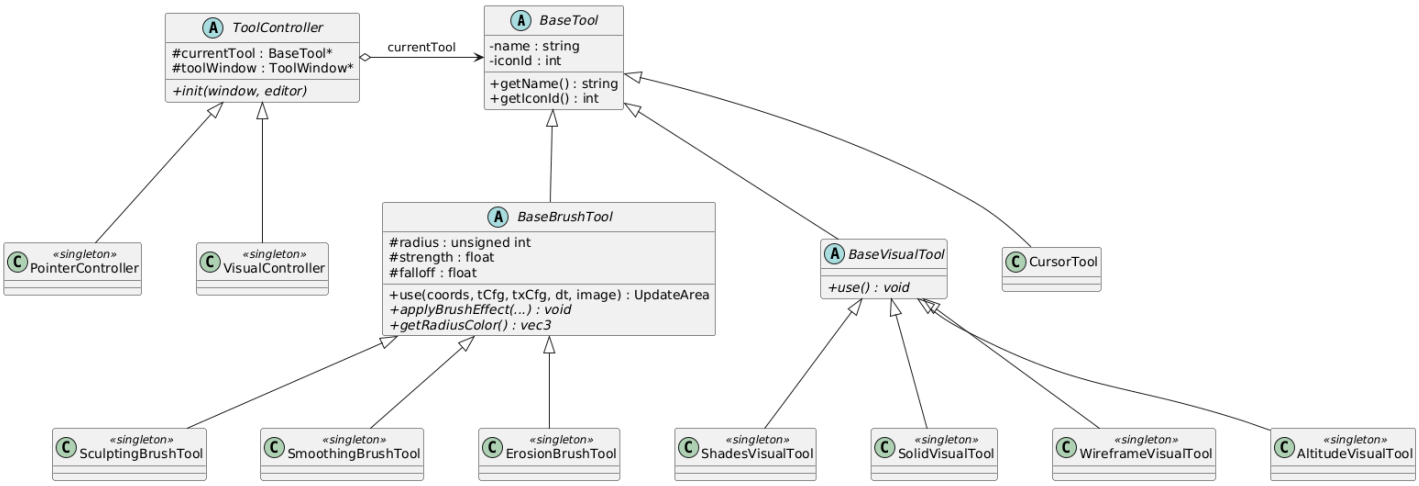


Figura A4: Gerarchia completa dei tool e pattern Strategy.

A5 — EventDispatcher (Observer)

Struttura del pattern Observer realizzato tramite la classe template singleton EventDispatcher.
I Publisher (TopMenuBar, InputController, SetupWindow) invocano dispatch(), i Subscriber (AppController, CameraGestureController, PointerController, VisualController) si registrano con subscribe().
Gli alias TopMenuDispatcher, MouseMoveDispatcher e MouseWheelDispatcher sono le istanze effettivamente usate nel codice.

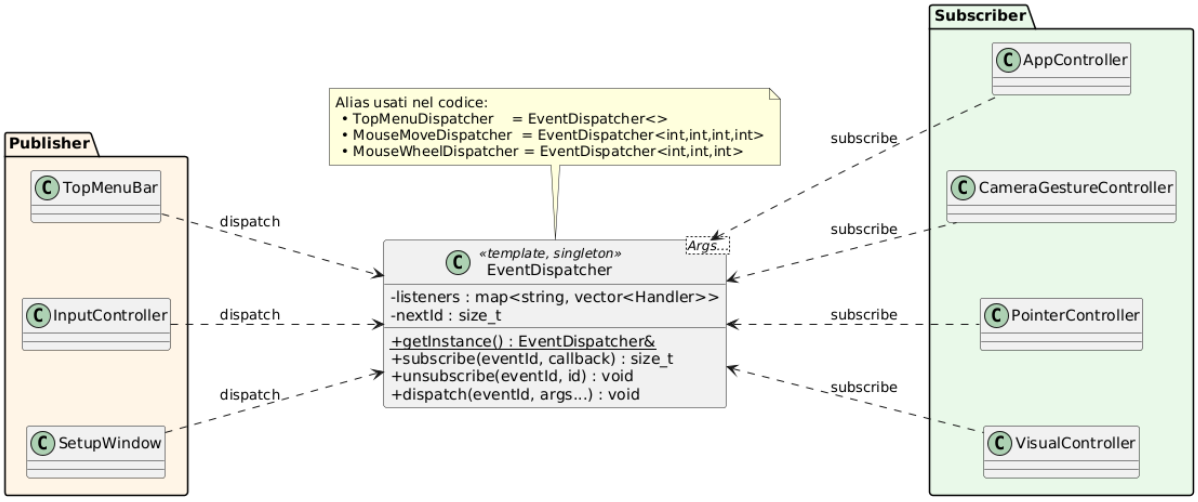


Figura A5: Pattern Observer realizzato dall'EventDispatcher.

STUDENTSUPSI

A6 — Sequence diagram: generazione del terreno

Sequence diagram che descrive il flusso completo di generazione del terreno a partire dal click su "Generate" nella SetupWindow che notifica al SetupController tramite l'interfaccia ISetupListener
L'esecuzione del task in background avviene da parte dell'AppController, la generazione della heightmap e della mesh avviene nei rispettivi generatori e infine parte il caricamento sul nodo root della scena dell'Engine.

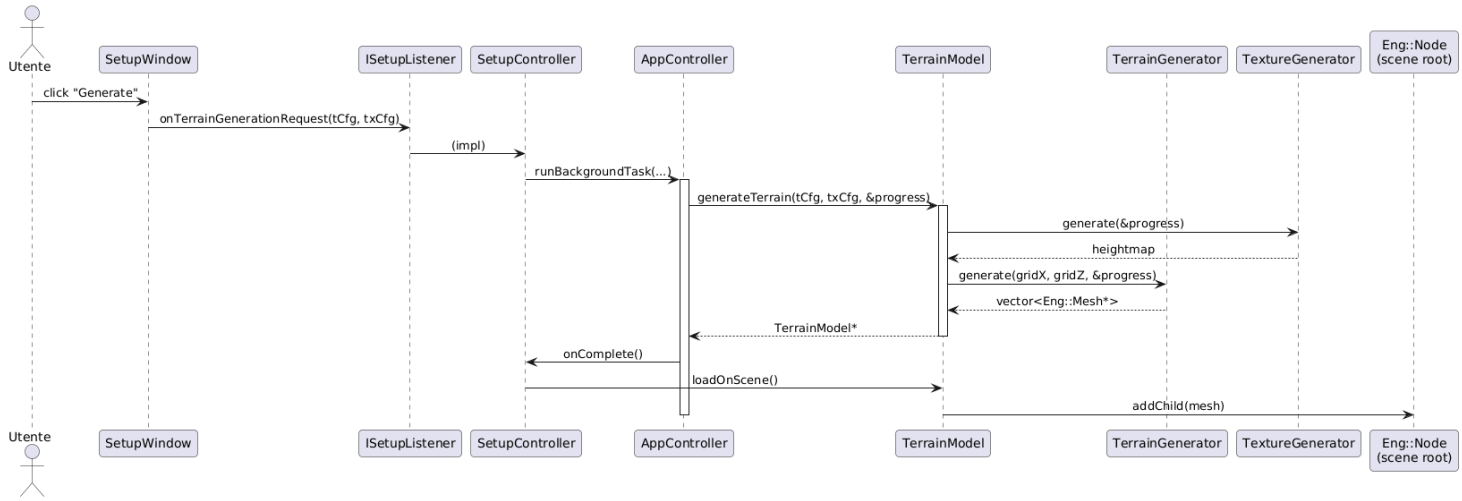


Figura A6: Sequence diagram del processo di generazione del terreno.

STUDENTSUPSI

B — Modalità di visualizzazione del terreno

- Le immagini seguenti mostrano lo stesso terreno renderizzato con le quattro modalità di visualizzazione descritte nel capitolo 3.1.
- Il fragment shader terrain.frag commuta tra le modalità in base a un uniform impostato dalla UI; la geometria e le normali sono identiche in tutte le viste.

Modalità Realistico

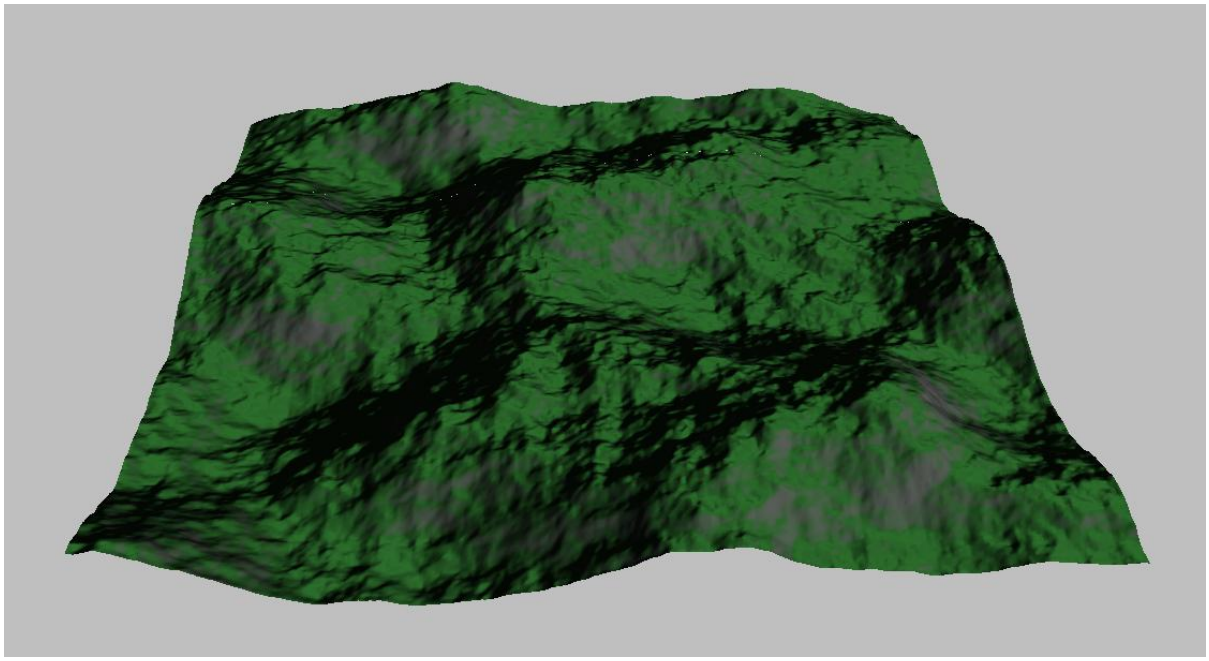
- 

Figura B1: Modalità Shading Realistico: Colore base derivato dalla pendenza (verde per zone pianeggianti, grigio per zone ripide) con illuminazione Blinn-Phong.

Modalità Solid

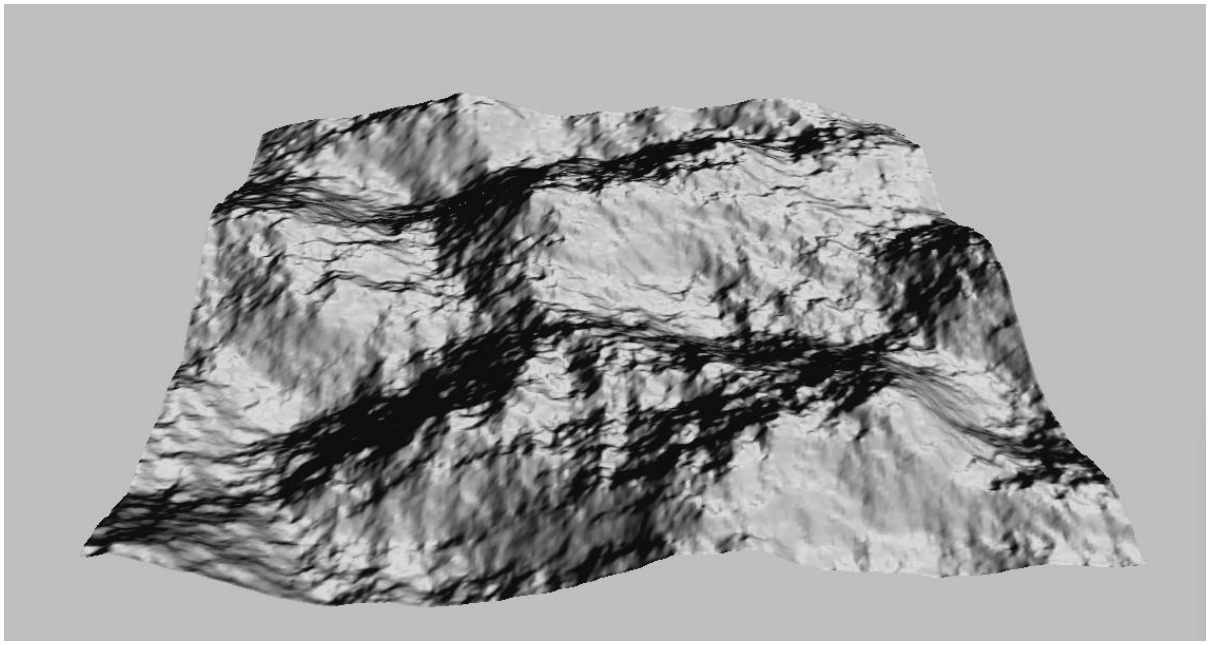


Figura B2: Modalità Solid: Colore uniforme che evidenzia la forma della mesh e le ombre prodotte dalle normali calcolate dinamicamente.

Modalità Wireframe

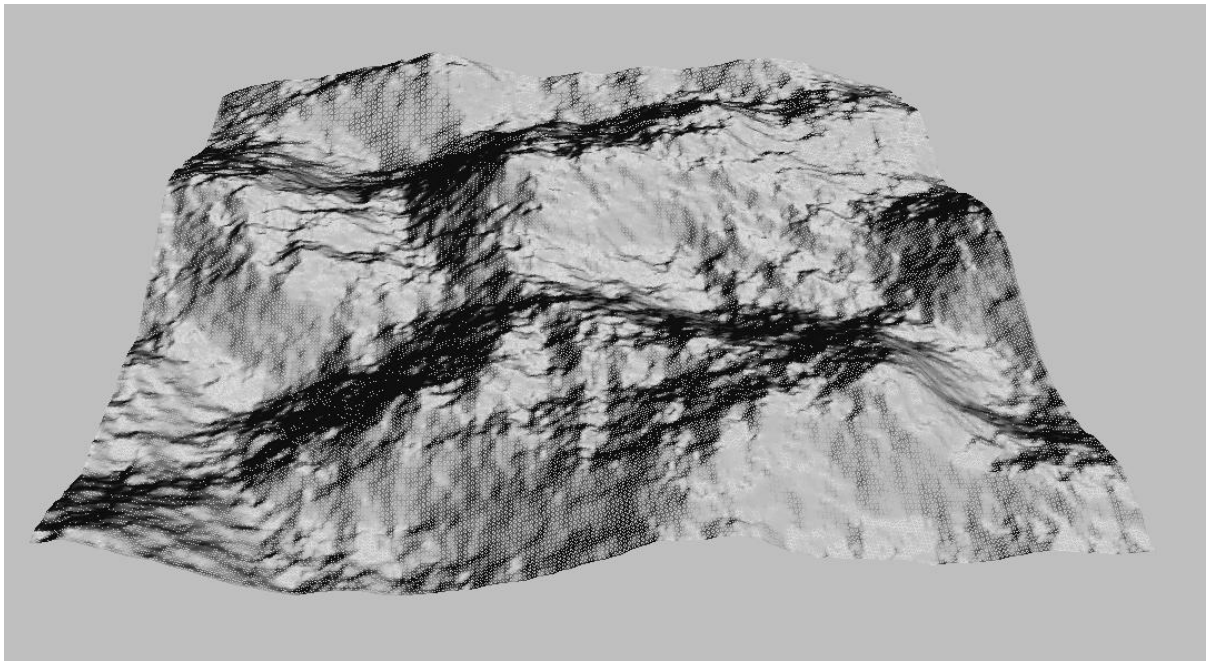


Figura B3: Modalità Wireframe: Visualizzazione dei soli spigoli, utile per analizzare struttura e LOD.

STUDENTSUPSI

Modalità Altimetrica

-

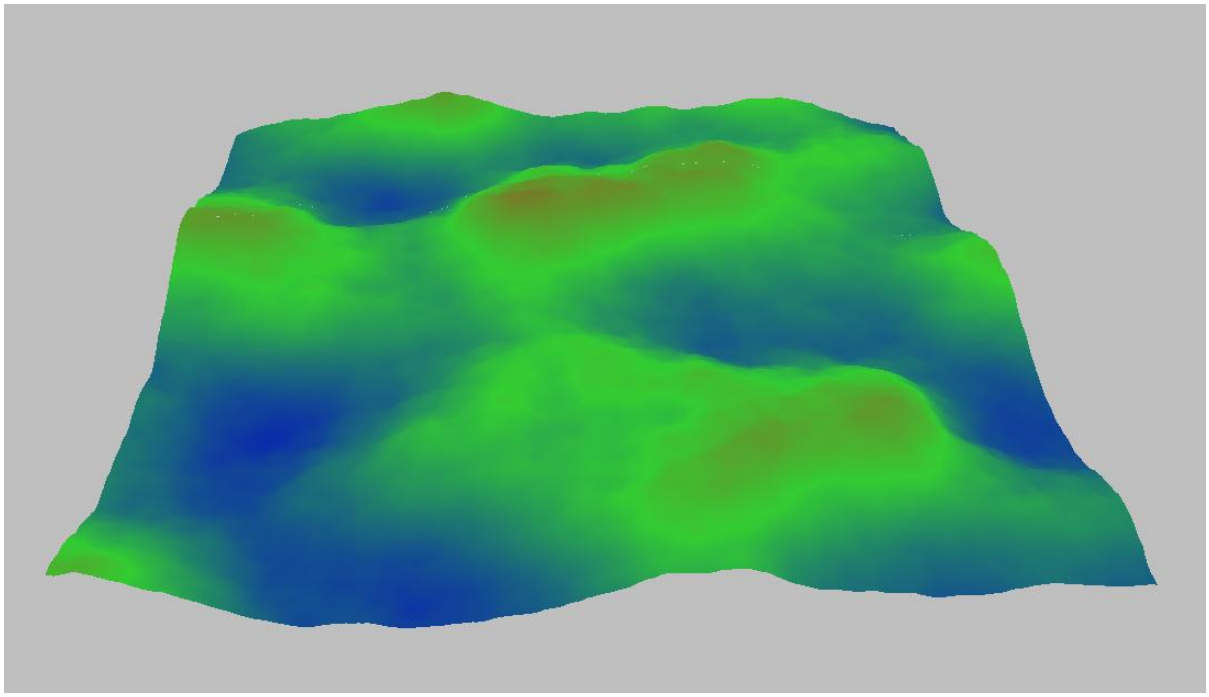


Figura B4: Modalità Altimetrica: Gradiente blu→verde→rosso basato sull'altezza normalizzata.

Allegati

Allegato A — Benchmark delle prestazioni

Documento separato (benchmark.docx) contenente le tabelle complete dei tempi di generazione del terreno e della texture al variare delle dimensioni, di style e della risoluzione (px/m). I dati sono riferiti all'analisi delle prestazioni discussa in §4.2.